

Determining Risk Thresholds in Credit Card Fraud Detection using Logistic Regression and Random Forests

Rishab Sakalkale, Henry Szklanny, Momin Shah, Praneet Bhagat

December 2025

1 Statement of Research Question

The guiding research question for this project is:

How can we find the optimal risk threshold for determining fraudulent card transactions that minimizes manual review costs (false positive classifications) while maintaining an acceptable level of fraud loss (false negative classifications)?

This question addresses both methodological and managerial concerns. From a methodological standpoint, logistic regression—the go-to classification model—is transparent and interpretable, but may struggle in highly imbalanced settings. From a business perspective, model interpretability, regulatory compliance, and actionable fraud-risk scoring are essential. We therefore examine whether threshold adjustment, regularization, and feature engineering allow classification methods to achieve practical levels of performance.

1.1 Feasibility of Addressing the Question

Answering the research question is feasible due to the richness and scale of the dataset. The full dataset contains over 13.3 million transaction-level records along with user-level demographic information and card-level attributes. Approximately 8.9 million of these transactions include fraud labels, of which 13,332 are fraudulent.

This dataset supports the modeling tasks for several reasons:

- Logistic regression requires a large number of observations for stable coefficient estimation, which the dataset provides in abundance.
- The dataset contains a diverse set of continuous, binary, and engineered features well-suited for elastic net regularization.
- Cross-validation, and threshold tuning are computationally feasible once the dataset is reduced to a manageable size via downsampling (3 million rows) while retaining all fraud cases.
- Random Forest modeling is feasible due to the reduced dataset size and the engineered feature set.

Together, these factors make this dataset ideal for evaluating both interpretable and more complex models for fraud detection.

1.2 Why the Dataset Qualifies as “Big Data”

This dataset satisfies several formal dimensions of “Big Data”—including volume, variety, and messiness—as used in financial analytics.

1.2.1 Volume

The dataset is large both in file size and in number of observations:

- Total data size exceeds 1.42 GB.
- Transaction file: 1.26 GB.
- Fraud label file (JSON): 160 MB.
- Total dimensions: 13,305,915 obs. $\times$ 12 features.

1.2.2 Variety

The dataset contains multiple heterogeneous sources:

- Transaction-level data (timestamps, amounts, merchant categories, chip usage)
- User-level demographic and geographic attributes
- Card-level information
- Fraud labels stored in JSON format

These sources differ in format, granularity, and structure and require several processing steps.

1.2.3 Messiness

As a real banking dataset, the raw data exhibit:

- missing values (e.g., ZIP codes, location fields),
- structural inconsistencies (e.g., negative card ages, zero credit limits),
- duplicate observations,
- categorical variables with extremely high cardinality (e.g., merchant category codes).

This necessitates extensive cleaning, validation, and feature engineering, consistent with Big Data workflows.

1.3 Organization of the Report

The remainder of this report is organized to provide a clear and logical progression from raw data to model interpretation and evaluation.

1. **Initial Discoveries and Data Merging.** We describe the four raw input files (transactional-level data, user-level data, card-level data, and fraud labels), outline the merging procedure, and summarize key exploratory findings such as the overall fraud rate and inherent class imbalance. This section establishes the scale and structure of the dataset that motivates later modeling decisions.
2. **Data Cleaning Challenges and Solutions.** We document the major inconsistencies detected in the raw data and our corresponding corrective actions. These include fixing variable structures, addressing missing ZIP codes, remediating negative card ages, handling zero credit limits, creating binary error indicators, removing duplicate rows, and justifying the treatment of high-cardinality variables such as Merchant Category Codes (MCC). This section emphasizes the importance of producing a reliable modeling dataset.
3. **Feature Engineering.** We introduce engineered variables that enhance predictive power, including time-based variables, location-type indicators, financial ratios, and cardholder-level transformations. A comprehensive data dictionary is provided to ensure transparency and reproducibility.
4. **Dataset Subsampling Strategy.** Given the computational constraints of the full 8.9 million labeled transactions, we justify the decision to undersample non-fraud cases to create a 3-million-row modeling dataset while retaining all fraud cases. We discuss the benefits and limitations of this approach for model training and generalization.
5. **Modeling Framework.** This section details the steps required to fit the logistic regression model using Elastic Net regularization and Random Forest classification model:
 - standardization of numeric predictors,
 - training/test stratification,
 - selection of α values,
 - 3-fold cross-validation to identify optimal λ ,

We evaluate the baseline model using Accuracy, Recall, Precision, ROC curves, and AUC.

6. **Threshold Optimization.** Because the default 0.5 threshold performs poorly in the presence of severe class imbalance, this section explores the Recall–Precision tradeoff and selects an optimal classification threshold (1.95%) based on business objectives. Updated confusion matrices and performance metrics are presented.
7. **Interpretation of Selected Features.** We examine the non-zero Elastic Net coefficients to identify which engineered and original features most strongly influence fraud likelihood. The direction, magnitude, and business relevance of each coefficient are interpreted.
8. **Random Forest Benchmark Model.** To contextualize the logistic regression results, we fit a Random Forest classifier to the same dataset. We compare performance metrics, ROC curves, and feature importance rankings between the two models, highlighting tradeoffs between interpretability and predictive performance.
9. **Summary of Findings.** We synthesize major insights: strong discriminative ability ($AUC \approx 0.93$), the necessity of threshold adjustment, improved Recall after tuning, and the importance of online, international, and high-credit-limit transactions. We also compare Logistic Regression to Random Forests in terms of business applicability.
10. **Limitations and Future Work.** We discuss limitations such as the loss of information due to undersampling, unmodeled temporal patterns, high-cardinality categorical variables, and the absence of methods such as SMOTE or richer interaction terms. Future extensions may include XGBoost, LightGBM, better treatment of MCCs, or advanced anomaly detection approaches.
11. **Conclusion.** The report concludes by addressing the original research question and evaluating whether logistic regression, enhanced by imbalance-mitigation techniques, provides a viable baseline fraud detection framework when compared to Random Forests.

2 Introduction to Logistic Regression

An objective of our project is to build a logistic regression model to predict fraudulent card transactions. Before we can even think about building a model, it is extremely necessary for us to first conduct pre-processing and cleaning of the datasets, combine them, and perform some preliminary feature engineering. Without going into extensive detail about every step, since this is not a course on data cleaning, we only provide the important details and discoveries of this process.

3 Initial Discoveries and Data Mergers

We use the card data, fraud labels, transaction data, and user data files. The transactional data CSV file contains transactional-level data with approximately 13.3 million rows (13,305,915 to be exact). The card data CSV file contains relevant card information. The user data CSV file contains information about the cardholder. And the labels is a JSON file that contains binary fraud labels for approximately 8.9 million transactions (8,901,631 to be exact). This means that the training set size is about 8.9 million, and the test size (i.e., the transactions without labels) is about 4.4 million, which corresponds to a 2/3:1/3 train-test split. For additional clarification, approximately 8.9 million of those 13.3 million transactions have fraud labels. It is on these 8.9 million that we will conduct a further train-test split using a conventional 80%–20% split. Another discovery we made while exploring the 8.9 million labeled transactions is that the fraud rate is $13,332/8,901,631 \approx 0.14955463\%$. This indicates that the data is highly imbalanced, and the model will probably perform poorly in detecting actual fraud cases. In addition to discovering the overall fraud rate, we were curious into seeing how this fraud rate varied by transaction type and the hour of day. Figure 1 shows that international transactions, almost regardless of hour of day, are the most indicative of fraud compared to domestic and online transactions. Domestic transactions are the least suspicious. In general, fraud cases seem to occur between the hours of 9 and 20—in other words, during normal hours people start work and are awake. Given how drastically different the fraud rate is according to hour and transaction type, we expect to see transaction type as a significant feature.

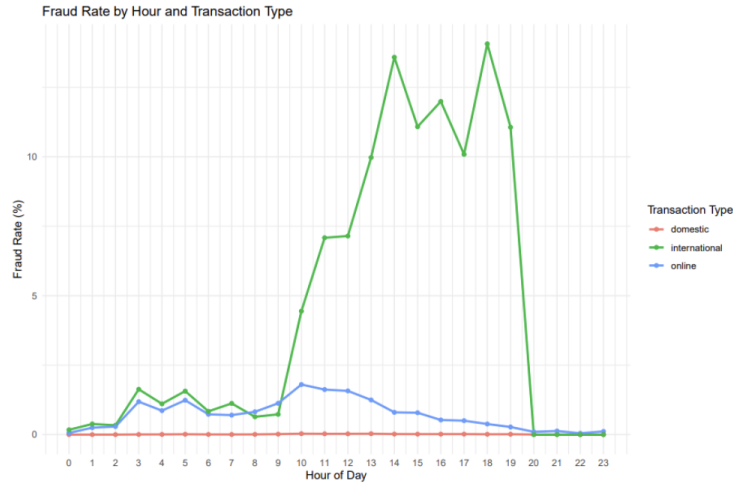


Figure 1: Fraud Rate by Hour and Transaction Type

Lastly, because the card data, fraud labels, transactional data, and user data files have been delivered as separate files, as part of the data pre-processing steps, we need to merge these files into one cohesive file to have all the features in one place.

4 Problems and Solutions During Cleaning

4.1 Variable Structure

The first issue we encountered is regarding the structure of some of the variables. The “amount” variable was being stored as a character with dollar signs, causing all comparisons to fail. During this fix, we also found that some of the amounts are negative. Positive amounts correspond to regular card expenses from the perspective of the buyer, while negative amounts correspond to refunds on the cards. We also found a small proportion of the amounts to be zero. A related issue that we discovered was that other financial columns were also being stored as characters. We simply removed the dollar signs and set their structure to numeric. We keep the negative and zero amounts.

4.2 Missing Zip Codes

Another issue we encountered was the absence of zip codes for some transactions. About 1.1 million zip codes were missing, accounting for 12.4% of all transactions. When investigated, it was discovered that most of these non-zip code transactions were predominantly online transactions. Understandably, the zip code in these cases would not be applicable. The other transactions with missing zip codes that are non-online are international transactions. Interestingly, when we investigated these 59,521 foreign transactions with missing zip codes, the fraud rate was roughly 5%. This is a critical data pre-processing discovery because it signals that perhaps international transactions are highly positively associated and have higher likelihood of being actual fraud cases compared to domestic transactions. Our solution to mitigate the issue of missing zip codes was simply to not use the variable in the model and to instead use location-type flags.

4.3 Negative Card Ages

Another issue we encountered was negative card ages. We discovered that 214 transactions had a card age that was less than zero days (i.e., the transaction occurred before the account was opened). We investigated the fraud rate among these cases and discovered that none of these 214 were fraud cases. The likely possible explanation is due to data entry error. To mitigate this issue, we set the card age for these transactions to be 0 (i.e., the transaction occurred on the same day the card was opened).

4.4 Zero Credit Limits

Another issue we encountered was zero credit limits. A credit limit is the maximum amount of money the bank or lender will allow the cardholder to borrow on the card. A zero credit limit theoretically renders the card useless.

35,673 of these were credit cards and 16 were debit cards. None of these cards were found on the dark web and the fraud rate was low, so it did not raise suspicions. There could be several reasons for this anomaly, but one likely possible explanation is that the card is new and has therefore not been issued a limit yet. Our solution to mitigate this issue was to create flags for cards with credit limits and set transaction percentages to zero when the limit is zero.

4.5 Errors Column Handling

Another issue we encountered was that the errors column was mostly empty text, some with multiple error types. The errors variable indicates the type of error encountered during a transaction. We decided to mitigate these mostly error-less transactions by creating a binary error indicator.

4.6 Duplicate Rows

We detected 14 duplicate rows and removed them. All duplicates were non-fraud cases.

4.7 Merchant Category Codes

We have chosen to keep MCC as a numeric variable rather than removing it entirely. This variable has high cardinality, which should have been reason for us to remove it as we had done with the other high cardinality variables, though not exclusively for this reason. MCC has 109 levels, each corresponding to a unique merchant code. The exploratory data analysis revealed substantial variation in fraud rates across different MCC codes—ranging from 0% to nearly 60%—indicating that MCC undoubtedly contains strong predictive signal for fraud detection, hence why we do not remove it despite its high cardinality (see Figure 2). Figure 2 does not tell the whole story, however, because a fraud rate of nearly 60% for the 4411 MCC, for example, is so high that it would raise considerable scrutiny from regulators—we need to examine the fractional proportion. High-fraud MCCs are rare but extremely risky. MCC 4411 has the highest fraud rate, but accounts for only roughly 0.0031% (276/8,914,949) of the entire data. Overall, the MCCs with fraud rate exceeding 10% account for only 2,830 transactions (0.0317% of the entire data). While treating MCC as numeric makes the assumption that similar code values represent similar merchant types, this approach is pragmatic given that one-hot encoding would create 109 additional columns and substantially increase model complexity. Using Elastic Net regularization is specifically designed to handle situations where predictive relationships are imperfect or features contain noise. By keeping MCC as numeric, we maintain the potential for the model to leverage this clearly informative variable while avoiding the computational burden and overfitting risk associated with high-dimensional encoding. The extreme cases for MCCs like 4411 and 5733 are so rare that one-hot encoding might not help much because the model will likely not have enough fraud examples to reliably learn that “MCC 4411 = fraud”. For the reader’s awareness, the makers of the dataset provide definitions of each merchant category code.

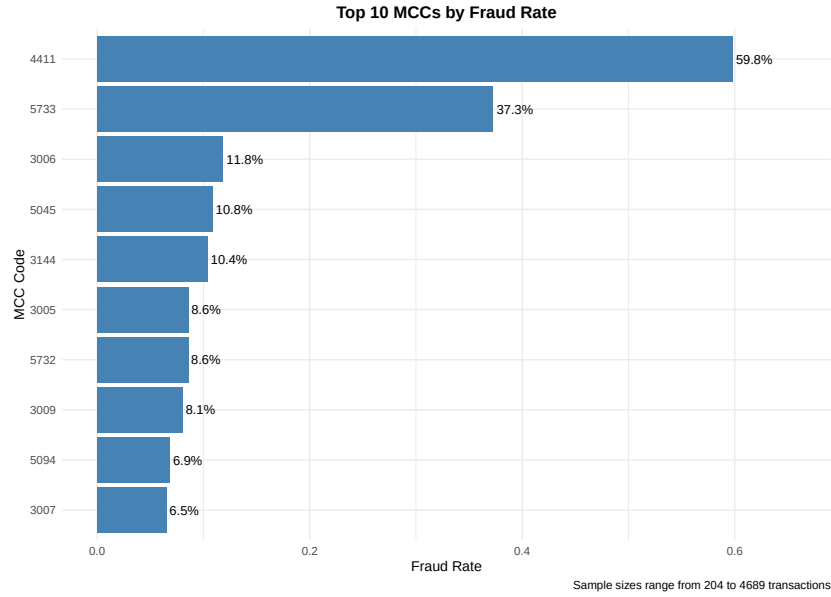


Figure 2: Top 10 MCCs by Fraud Rate

5 Engineered Features

As part of the data pre-processing steps, we engineer some important features that may have predictive power in the classification task. Although the standalone features that came with the datasets are valuable, we deem that engineering other features, especially financial ratios, could help detect potential fraud. These include amount features, time-based features, location features, card features, financial ratio features, and user features.

6 Data Dictionary

6.1 Dataset Overview

- **Dataset Name:** train_clean_processed
- **Observations:** 8,914,949 (labeled)
- **Variables:** 35 (18 original and 17 engineered)
- **Target Variable:** fraud_label
- **Fraud Rate:** $\approx 0.14955\%$ (13,332 fraud cases)

6.2 Original Variables from Source Data

Table 1: Original Variables - Transaction and Card Features

#	Variable	Type	Description	Source
1	fraud_label	Binary	Target variable: Fraud indicator (1 = fraud, 0 = legitimate)	Labels JSON
2	use_chip	Factor	Type of transaction method	transactions_data.csv
3	mcc	Numeric	Merchant Category Code (109 unique values)	transactions_data.csv
4	card_brand	Factor	Credit card brand/network	cards_data.csv
5	card_type	Factor	Type of card account	cards_data.csv
6	has_chip	Factor	Whether physical card has a chip	cards_data.csv
7	num_cards_issued	Numeric	Number of cards issued to user	cards_data.csv
8	credit_limit	Numeric	Credit limit on card (dollars)	cards_data.csv
9	year_pin_last_changed	Numeric	Year PIN was last changed	cards_data.csv

Table 2: Original Variables - Cardholder Demographics and Financial

#	Variable	Type	Description	Source
10	current_age	Numeric	Current age of cardholder (years)	users_data.csv
11	retirement_age	Numeric	Expected retirement age	users_data.csv
12	birth_month	Numeric	Month of birth (1-12)	users_data.csv
13	gender	Factor	Gender of cardholder	users_data.csv
14	per_capita_income	Numeric	Avg income per person in area (dollars)	users_data.csv
15	yearly_income	Numeric	Annual income of cardholder (dollars)	users_data.csv
16	total_debt	Numeric	Total debt of cardholder (dollars)	users_data.csv
17	credit_score	Numeric	Credit score (300-850)	users_data.csv
18	num_credit_cards	Numeric	Number of credit cards owned	users_data.csv

6.3 Engineered Features

Table 3: Engineered Features - Time and Location

#	Variable	Type	Description	Formula/Logic
19	hour	Numeric	Hour of day (0-23, 24-hour format)	Extracted from date
20	day_of_week	Numeric	Day of week (1=Sunday, 7=Saturday)	Extracted from date
21	is_weekend	Logical	Weekend transaction flag	day_of_week $\in$ {1, 7}
22	month	Numeric	Month of transaction (1-12)	Extracted from date
23	day_of_month	Numeric	Day of month (1-31)	Extracted from date
24	card_age_days	Numeric	Days since card account opened	date - acct_open_date
25	is_online	Logical	Online transaction flag	merchant_city == "ON-LINE"
26	is_international	Logical	International (non-US) transaction	Missing zip AND not on-line
27	transaction_type	Factor	Location type of transaction	Combines is_online & is_international

Table 4: Engineered Features - Transaction Amount and Financial Ratios

#	Variable	Type	Description	Formula/Logic
28	amount_abs	Numeric	Absolute value of transaction (dollars)	abs(amount)
29	transaction_direction	Factor	Direction of money flow	Based on amount sign
30	debt_to_income	Numeric	Ratio of total debt to yearly income	total_debt / yearly_income
31	transaction_pct_limit	Numeric	Transaction as % of credit limit	(amount_abs / credit_limit) $\times$ 100
32	transaction_pct_income	Numeric	Transaction as % of yearly income	(amount_abs / yearly_income) $\times$ 100
33	has_error	Binary	Transaction had errors flag	errors $\neq$ ""
34	years_to_retirement	Numeric	Years until retirement age	retirement_age - current_age
35	has_credit_limit	Logical	Card has credit limit > 0	credit_limit > 0

6.4 Categorical Variable Levels (One-Hot Encoded)

Table 5: Categorical Variable Levels (One-Hot Encoded)

Variable	# Levels	Levels
use_chip	3	Chip Transaction, Online Transaction, Swipe Transaction
card_brand	4	Amex, Discover, Mastercard, Visa
card_type	3	Credit, Debit, Debit (Prepaid)
has_chip	2	NO, YES
gender	2	Female, Male
transaction_type	3	domestic, international, online
transaction_direction	3	purchase, refund, zero
fraud_label	2	0 (no fraud), 1 (fraud)

6.5 Logical Variable Definitions (One-Hot Encoded)

Table 6: Logical (TRUE/FALSE) Variables (One-Hot Encoded)

Variable	TRUE =	FALSE =
is_weekend	Weekend transaction	Weekday transaction
is_online	Online transaction	Physical location transaction
is_international	International (non-US)	US transaction
has_error	Transaction had errors	No errors
has_credit_limit	Has credit limit > 0	Credit limit = 0

6.6 Variable Groups by Category

Table 7: Variables Grouped by Purpose

Category	Variables
Transaction Characteristics	use_chip, mcc, amount_abs, transaction_direction, has_error
Time Features	hour, day_of_week, is_weekend, month, day_of_month
Location Features	is_online, is_international, transaction_type
Card Features	card_brand, card_type, has_chip, num_cards_issued, credit_limit, has_credit_limit, card_age_days, year_pin_last_changed
Cardholder Demographics	current_age, retirement_age, birth_year, birth_month, gender, years_to_retirement
Financial Features	yearly_income, total_debt, credit_score, num_credit_cards, per_capita_income
Risk Ratios (Engineered)	debt_to_income, transaction_pct_limit, transaction_pct_income
Target Variable	fraud_label

6.7 Removed Variables

The following variables were present in the original datasets but removed during data cleaning:

Table 8: Dropped Variables and Reasons

Reason	Variables
ID Variables (cause overfitting)	id, client_id, card_id, merchant_id, client_id_card
Sensitive Information	card_number, cvv, address
Redundant (replaced by engineered features)	date, amount, errors, zip, merchant_city, acct_open_date, date_parsed
High Cardinality (too many levels)	merchant_state (200 levels), latitude (740 values), longitude (859 values), expires
No predictive power because only one level	card_on_dark_web
Perfectly correlated with Another Variable	birth_year (correlated with current_age)

7 Brief Exploratory Data Analysis

We performed some brief EDA on the data containing the variables from the data dictionary in addition to our initial findings in constructing the training dataset. First, we wanted to see if a small subset of users accounted for a majority of the fraudulent transactions:

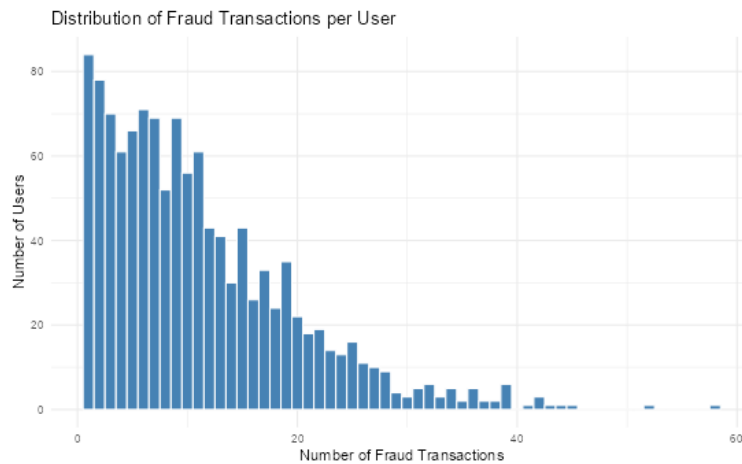


Figure 3: Fraud Count per User

Out of the 2000 total users, the max fraudulent transaction for a single user is about 60, far lower than the total fraudulent transaction count of 13,332. This signals there are no extreme outliers in user data, a good check of data quality. We also want to examine the transaction amounts as a percentage of credit limits and yearly income, plotted below:

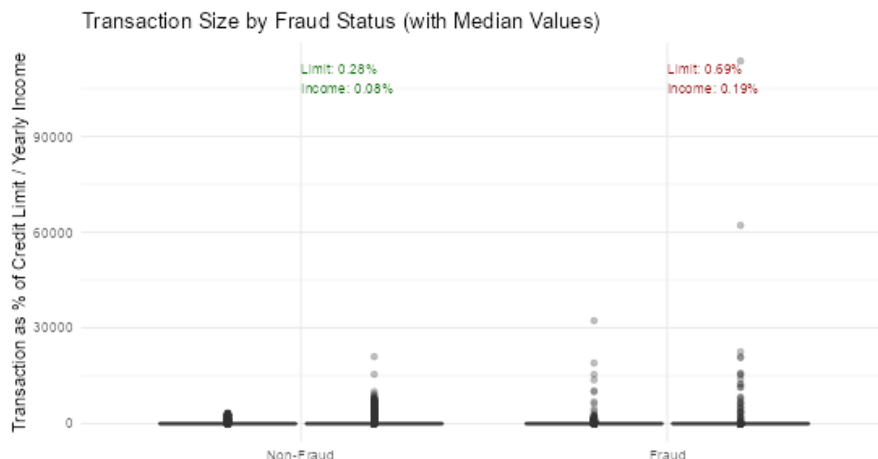


Figure 4: Transaction Amount as Percentage of Credit Limit and Yearly Income

To clarify the format of the plot above, the left box plot for the non-fraud and fraud cases is the amount as a percentage of credit limit, and the right box plot is the amount as a percentage of yearly income. The median percent of credit limit and yearly income for the non-fraud transactions is 0.28% and 0.08% respectively, and 0.69% and 0.19% for fraudulent transactions. We expect that these two risk ratios (which we include as engineered features) will be selected as influential variables in our models.

Model-Making and Metric Evaluation

8 Initial Dataset Undersampling

The cleaned and pre-processed dataset we call `train_clean_processed` has a total 8,914,949 transactional observations with 35 features (18 original and 17 engineered). In this dataset, there are a total of 13,332 fraud cases and the rest are non-fraud cases. We decided to reduce the dataset to 3 million observations while retaining all actual fraud cases to balance computational efficiency with model integrity. Undersampling was done by choosing non-fraud rows uniformly at random. The initial dataset is extremely large, which has shown to be computationally intensive and time-consuming for model-making and cross-validation. By undersampling the non-fraud cases while keeping all the fraud cases, we ensure that the model still learns from all available examples, which are critical for detecting fraud. With this undersampled distribution, the overall fraud rate is now 0.44%. We believe this approach preserves the class distribution for the minority class and prevents the model from losing important information about actual fraud, while reducing training time and memory requirements. This approach isn't without drawbacks, however. We believe possible drawbacks include loss of information from non-fraud cases. By undersampling the non-fraud observations, there may be loss of subtle patterns present in the full dataset even when choosing non-fraud rows uniformly at random. Moreover, if the 3 million observations are not perfectly representative of the initial full dataset, the model might suffer from overfitting to the subsample.

9 Training Stratification

The undersampled dataset has a total of 3 million observations of which 13,332 are actual fraud and the rest are non-fraud labels. We perform a conventional 80%-20% train-test split, meaning 2.4M transactions are for testing and 0.6M are for testing. These are split into training and testing sets. In the training set (i.e., out of those 2.4M observations), 2,389,382 are non-fraud cases and 10,618 are fraud-cases.

10 Standardizing the Features

Before proceeding with Elastic Net Regularization, we standardize the numeric continuous features (excludes logical and one-hot encoded features). This is conventional procedure when using Elastic Net. We fit the scale on the

training data only and then apply to both train and test sets.

11 Elastic Net Regularization and Performance

We use the `glmnet` package in R. The `glmnet` package is very efficient for fitting Elastic Net Regularization on logistic regression models, which motivates us to use it. The number of predictors we have is many and Elastic Net is also good for dealing with unaddressed or undetected correlations in the features during pre-processing. The hyperparameter of interest is $\alpha \in [0, 1]$, which determines the mix between L_1 and L_2 penalties. The choice of α is important because its value affects the degree of shrinkage the coefficients on our predictors get. We prepare the feature matrix and response vector for the train set and do the same for the test set. We created a vector of α values to loop over to test different L_1 and L_2 Elastic Net mixtures. The vector is

$$\alpha = \begin{bmatrix} 0.25 \\ 0.50 \\ 0.75 \end{bmatrix}$$

The reason we use only 3 α values is to be more computationally efficient. Testing 3 α values using 3-fold CV already took about an hour for the script to run. We then use cross-validation. Each α has its own cross-validation over a grid of λ values. Again, we resort to 3-fold instead of 5-fold CV to save time. One hour wait time is expected because with 3 million rows in the training set and Elastic Net with CV, 3-fold is fitting the model 3 times and `cv.glmnet` is running about 100 different λ values. All this means is that it's testing about 300 models per α value. It is easy to see then if we were to keep the initial 8.9 million rows, run 5-fold CV instead, and try a larger range of α values, it would be a very inefficient and time-consuming process. Once complete, the combination producing the high test AUC is the one we choose. The CV optimizing metric we use is AUC and we set the family argument to binomial. Below we present the results of Elastic Net when predictions were made on the test set.

Table 9: Model Comparison for α Values

α	CV_AUC	Test_AUC	Test_Accuracy	Best λ
0.25	0.9330461	0.9307499	0.9954850	0.000770615
0.50	0.9330901	0.9302250	0.9954883	0.0003853075
0.75	0.9330535	0.9314992	0.9954867	0.001370847

The best parameter from 3-fold CV is $\alpha = 0.75$ because it produces the largest AUC, which was our optimization metric. This alpha value means the model behaves more like LASSO, which is why the λ value is the highest. The accuracies are all very high, but this is a misleading statistic due to the high imbalance. Below we provide the confusion matrix of the model for $\alpha = 0.75$ ($\lambda=0.001370847$).

		Actual	
		0	1
Predicted	0	597,269 (TN)	2,691 (FN)
	1	17 (FP)	23 (TP)

The accuracy was already computed as 99.54867%. Computing the Recall is easy. It computed as

$$\text{Recall} := \frac{TP}{TP + FN} \quad (1)$$

The Recall is 0.847%. The Recall is abysmal compared to the Accuracy. Recall is the metric we care more about because we want to capture as many positive (fraud) cases as possible. Below we also provide the ROC Curve.

What is to be understood by these metrics? An accuracy of about 99% means the model predicts “not fraud” for almost all transactions. A recall that is less than 1% means the model is catching almost none of the actual fraud cases. An AUC of about 0.93 means the model can distinguish fraud from non-fraud in terms of probabilities, but the default 0.5 threshold is inappropriate. The threshold needs to come down even if that means at the expense of accuracy. We should attempt to find the optimal threshold for maximum Recall.

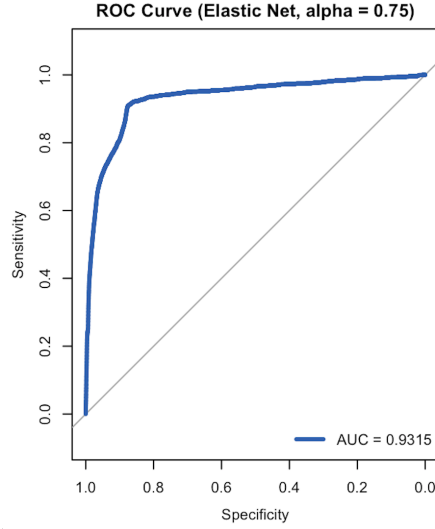


Figure 5: ROC Curve

12 Finding a Recall-Optimizing Threshold

It is reassuring to see the AUC as 0.93. At the beginning of this project we thought we would resort to using the Synthetic Minority Oversampling Technique (SMOTE) to address the imbalance issue. Although tuning the threshold and SMOTE are both valid approaches, they solve different problems at different stages and have different tradeoffs. We already have a good model. The AUC metric of 0.93 means the model is distinguishing fraud from non-fraud. We just need to adjust where to draw the line. Threshold tuning is easier compared to SMOTE because there is no retraining required or retuning of the α parameter. The threshold is also more interpretable. We can easily adjust the threshold based on business needs. We now evaluate Recall and Precision across many thresholds, starting at 0 and going to 0.20 by 0.0005 increments. According to our findings, the best threshold is 1.95% and the optimized Recall at this threshold is 73.28666%. The selected threshold of 1.95% means that any transaction with predicted probability equal to or exceeding 1.95% is classified as fraud. We are now catching 73% of all fraud cases in the test set, which is a major improvement. The Precision is now 5.304%.

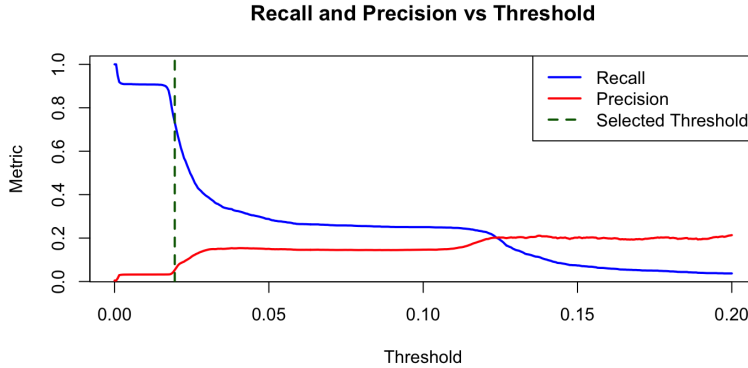


Figure 6: Recall and Precision vs Threshold

As seen in Figure 3, as the threshold increases, Recall increases and Precision decreases. The trade-off is that now more non-fraud cases are flagged. However, it is more important to correctly identify actual fraud cases than to avoid misclassifying non-fraud cases as fraud. Additionally, the new accuracy with this new threshold is 93.96%. A lower accuracy is another trade-off because many more transactions are now flagged as fraudulent. The number of false positives increases. Below is the new confusion matrix.

		Actual	
		0	1
Predicted	0	561,775 (TN)	725 (FN)
	1	35,511 (FP)	1,989 (TP)

13 Selected Features from Elastic Net

The non-zero coefficients that remained in the model are

Table 10: Important Features

Feature	Coefficient
transaction_type.domestic	-2.35161385
is_international	1.52795792
transaction_type.international	1.21522347
use_chip.Online Transaction	0.41115567
amount_abs	0.18404807
is_online	0.15734133
transaction_type.online	0.13192733
use_chip.Swipe Transaction	-0.08463287
transaction_pct.limit	0.08447891
per_capita_income	-0.046222870

These are Elastic Net coefficients, so they represent the log-odds of each feature on predicting fraud. Positive coefficients increase the likelihood of fraud, negative coefficients decrease it. Domestic transactions are much less likely to be fraud. A transaction being international increases the risk of fraud. Higher absolute transaction amounts slightly increase the likelihood of fraud. Swipe transactions are slightly less likely to be fraud compared to other types. `transaction_pct.limit` is the transaction as the percentage of the card’s credit limit. The positive coefficient on this feature means that higher percentage of limit used increases fraud risk. `per_capita_income` is the average income per person in the area. Higher per-capita income of the cardholder slightly reduces fraud risk. We also provide a helpful bar chart below showing the coefficient magnitudes of each feature.

14 Summary of Logistic Regression Results

Our Elastic Net logistic regression model showed strong discriminative performance despite heavy class imbalance. We used a reduced but still representative dataset of 3 million transactions. We retained all fraud cases. The model we built has high discriminative ability, that is, that AUC metric is around 93%. The larger the area under the curve, the better. Even with extreme class imbalance, our model is able to rank fraud versus non-fraud transaction very well. This high AUC is one strength of our model. However, the initial and default 0.5 threshold was very low, which shows how class imbalance causes the model to favor predicting the majority class. We tuned this threshold down to 1.95%. In doing so, we substantially improved Recall to about 73%. This is a remarkable statistic because it means that our model is now identifying a meaningful proportion of fraud cases, but at the cost of increased false positives. In other words, the sacrifice for a higher Recall is a much lower Precision. But for banks, it is more important to identify actual fraud cases. To that end, we suggest a possible decision of setting the threshold near 1.95% if the goal is strictly to maximize catching fraud, not to find a balance between Recall and Precision. Furthermore, after adjusting this threshold, accuracy decreased to about 94%. The most important features of our model for predicting fraud include if the transaction is domestic or international, if it occurs online or not, the absolute value, if it was a swipe transaction or not, the percentage of the card’s credit limit used, and per-capita income. We initially suspected that using the SMOTE technique might have been favorable to bring the minority class to a better level of representation by oversampling so that the model can learn actual fraud cases. Another strength of the logistic regression model is the fact that we are able to produce a final output of interpretable coefficients. We can see which features increase or decrease fraud risk.

15 Using Random Forest

While tuned for recall, the results of the Logistic Regression model may not be good enough for banking applications, since there is still a high false negative and false positive rate. This motivates us to use a more advanced classification model that is better equipped to handle complex features - the Random Forest Classifier. This model, while not covered in class, is based on the simple weak learner, the decision tree, whose nodes represent decisions made based on features. Random Forest models amplify this by constructing many trees, randomly assigning subsets of features to tree to promote diversity, then combines the results of the trees to provide a final classification - a process called bagging. Due to this process, the model performs better with complex data while avoiding overfitting, making it an appropriate choice when considering alternative models to logistic regression.

Using the same 35 features and 80%-20% train-test split as the logistic regression model, we train the Random Forest Classifier, and get the following results on the test dataset (with a decision threshold of $p > 0.5$):

Predicted	Actual	
	0	1
	0	597,308 (TN) 1604 (FN)
1	26 (FP)	1063 (TP)

While not directly interpretable, variable importance scores can be calculated using permutation importance:

Table 11: Important Features

Feature	Coefficient
mcc	17679.471
transaction_pct_limit	4511.160
amount_abs	3780.408
transaction_pct_income	3616.614
hour	2648.508
per_capita_income	2352.517
card_age_days	2088.105
yearly_income	2022.231
credit_limit	2010.930
transaction_type.international	2004.995

This model has an out-of-box accuracy of 99.73% and a precision rate of 97.61%, but a very low Recall rate of 39.85%. This is opposite to the goals of a banking institution, so we need to tune this decision threshold, which we can do by inspecting the False Negative and False Positive rates (FNR and FPR respectively), seen in the plot below as a function of the decision threshold.

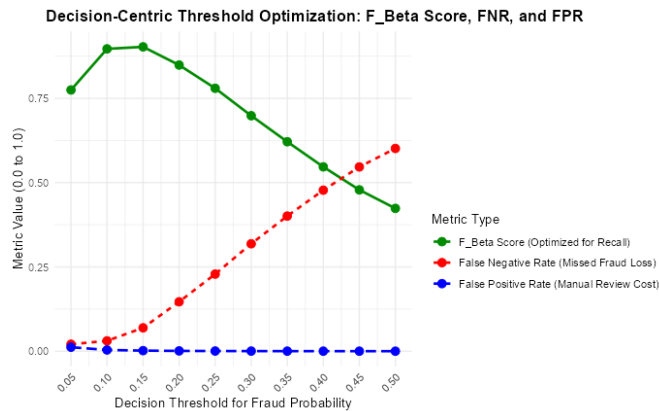


Figure 7: RF Model FNR and FPR Thresholds

In Figure 5, we notice a steep increase in FNR, or missed fraud loss, once the decision threshold becomes greater than 0.15. However, below 0.10, the FPR, or manual review cost, increases. To find the optimal threshold that balances the two rates, we can use the F_β score, which balances Recall and Precision, weighing the former with a factor proportional to β . This β value must be decided with the business interests of banks in mind, but for this report we use a β value of 3. We see from the plot that the highest F_β scores hover around 0.10 to 0.15. So, based on the F_β scores and the FNR/FPR rates, we can empirically, we can determine an ideal decision threshold of $p > 0.10$. Now, we get a confusion matrix on the test dataset of:

		Actual	
		0	1
Predicted	0	595,100 (TN)	82 (FN)
	1	2,234 (FP)	2,585 (TP)

This result is a big improvement: the accuracy dips slightly to 99.614%, but Recall jumps to 96.92% and Precision increases to 53.64%. The effect of using the F_β score to tune this is clear, while Precision is still weighted in the score and does see an improvement, a vast majority of performance gain comes in the Recall score. This is exactly what is required of a fraud detection algorithm in banking institutions.

16 Conclusions, Limitations, and Future Work

Our main research question was “How can we find the optimal risk threshold for determining fraudulent transactions that minimizes manual review costs (false positive classifications) while maintaining an acceptable level of fraud loss (false negative classifications)?”. Overall, our results suggest that the Elastic Net model serves as an effective baseline approach, while the Random Forest shows improvement in Recall by tuning decision thresholds via the F_β score. Throughout this project, our main objective was to maximize Recall as implied in our research question, even if that meant sacrificing Precision. Our research question focused on identifying as many true fraud cases as possible, rather than minimizing false alarms. For this reason, future work should explore models that achieve a better tradeoff between Recall and Precision so that the model does not need to sacrifice Precision as heavily as we did. Despite this limitation, our models achieved a high Recall after threshold adjustment, making it a decent baseline for identifying actual fraud cases. Although we initially considered using SMOTE, the results showed that threshold tuning alone was sufficient for improving the model’s ability to detect fraud.

For the sake of having a more scalable approach, we use a subset of transactions and sparse matrices to make computations more feasible. Although subset sampling made our computations easier and time-consuming by reducing the data down to 3 million transactions, we acknowledge that using 3 million instead of the full 8.9 million may have ignored some patterns in the unsampled non-fraud cases. Another limitation in our study is that we did not fully explore temporal patterns like seasonality or time-series trends.

For future work, we suggest trying more suitable models like XGBoost or LightGBM. Although these models are more advanced and not covered in this course, they are more suitable for datasets with heavy class imbalances. Also, while we did not use SMOTE, it would be interesting to see SMOTE used for better Precision/Recall tradeoffs. If logistic regression is to be tried again, we also recommend using an interaction term between the hour variable and transaction type variable. As we mentioned in the initial discoveries, we expected the transaction type variable to be a significant feature, and it did appear to be in both the logistic regression and RF models. We believe, however, inclusion of such an interaction would allow the model to learn distinct hour effects for each transaction type. This approach would first of all acknowledge the timezone limitation while secondly leveraging strong fraud signals presents in temporal data, particularly from international transactions. We also recommend for future direction that the MCC variable be treated as categorical instead of numeric, but perhaps using only the top 10 MCCs according to highest fraud rate. This approach would prevent computational burden and overfitting risk associated with high-dimensional encoding, and possibly move MCC into the list of important features.